

Forget to remember
Remember to forget

Long Short Term Memories and Gated Recurrent Units

Jonathon Hare

Vision, Learning and Control
University of Southampton

Some of the images and animations used here were originally designed by Adam Prügel-Bennett.

Recap: An RNN is just a recursive function invocation

- $\mathbf{y}(t) = \mathbf{f}(\mathbf{x}(t), \mathbf{c}(t) | \mathbf{W}_f)$
- and the state $\mathbf{c}(t) = \mathbf{g}(\mathbf{x}(t), \mathbf{c}(t-1) | \mathbf{W}_g)$
- If the output $\mathbf{y}(t)$ depends on the input $\mathbf{x}(t-2)$, then prediction will be

$$\mathbf{f}(\mathbf{x}(t), \mathbf{g}(\mathbf{x}(t), \mathbf{g}(\mathbf{x}(t-1), \mathbf{g}(\mathbf{x}(t-2), \mathbf{c}(t-2) | \mathbf{W}_g) | \mathbf{W}_g) | \mathbf{W}_g) | \mathbf{W}_f)$$

- it should be clear that the gradients of this with respect to the weights can be found with the chain rule

Recap: An RNN is just a recursive function invocation

- $\mathbf{y}(t) = \mathbf{f}(\mathbf{x}(t), \mathbf{c}(t) | \mathbf{W}_f)$
- and the state $\mathbf{c}(t) = \mathbf{g}(\mathbf{x}(t), \mathbf{c}(t-1) | \mathbf{W}_g)$
- If the output $\mathbf{y}(t)$ depends on the input $\mathbf{x}(t-2)$, then prediction will be

$$\mathbf{f}(\mathbf{x}(t), \mathbf{g}(\mathbf{x}(t), \mathbf{g}(\mathbf{x}(t-1), \mathbf{g}(\mathbf{x}(t-2), \mathbf{c}(t-2) | \mathbf{W}_g) | \mathbf{W}_g) | \mathbf{W}_g) | \mathbf{W}_f)$$

- it should be clear that the gradients of this with respect to the weights can be found with the chain rule
- The back-propagated error will involve applying $\mathbf{f}$ multiple times

Recap: An RNN is just a recursive function invocation

- $\mathbf{y}(t) = \mathbf{f}(\mathbf{x}(t), \mathbf{c}(t) | \mathbf{W}_f)$
- and the state $\mathbf{c}(t) = \mathbf{g}(\mathbf{x}(t), \mathbf{c}(t-1) | \mathbf{W}_g)$
- If the output $\mathbf{y}(t)$ depends on the input $\mathbf{x}(t-2)$, then prediction will be

$$\mathbf{f}(\mathbf{x}(t), \mathbf{g}(\mathbf{x}(t), \mathbf{g}(\mathbf{x}(t-1), \mathbf{g}(\mathbf{x}(t-2), \mathbf{c}(t-2) | \mathbf{W}_g) | \mathbf{W}_g) | \mathbf{W}_g) | \mathbf{W}_f)$$

- it should be clear that the gradients of this with respect to the weights can be found with the chain rule
- The back-propagated error will involve applying $\mathbf{f}$ multiple times
- Each time the error will get multiplied by some factor a

Recap: An RNN is just a recursive function invocation

- $\mathbf{y}(t) = \mathbf{f}(\mathbf{x}(t), \mathbf{c}(t) | \mathbf{W}_f)$
- and the state $\mathbf{c}(t) = \mathbf{g}(\mathbf{x}(t), \mathbf{c}(t-1) | \mathbf{W}_g)$
- If the output $\mathbf{y}(t)$ depends on the input $\mathbf{x}(t-2)$, then prediction will be

$$\mathbf{f}(\mathbf{x}(t), \mathbf{g}(\mathbf{x}(t), \mathbf{g}(\mathbf{x}(t-1), \mathbf{g}(\mathbf{x}(t-2), \mathbf{c}(t-2) | \mathbf{W}_g) | \mathbf{W}_g) | \mathbf{W}_g) | \mathbf{W}_f)$$

- it should be clear that the gradients of this with respect to the weights can be found with the chain rule
- The back-propagated error will involve applying $\mathbf{f}$ multiple times
- Each time the error will get multiplied by some factor a
- If $\mathbf{y}(t)$ depends on the input $\mathbf{x}(t-\tau)$ then the back-propagated signal will be proportional to $a^{\tau-1}$

Recap: An RNN is just a recursive function invocation

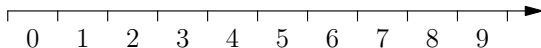
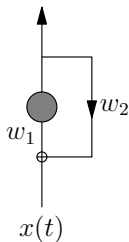
- $\mathbf{y}(t) = \mathbf{f}(\mathbf{x}(t), \mathbf{c}(t) | \mathbf{W}_f)$
- and the state $\mathbf{c}(t) = \mathbf{g}(\mathbf{x}(t), \mathbf{c}(t-1) | \mathbf{W}_g)$
- If the output $\mathbf{y}(t)$ depends on the input $\mathbf{x}(t-2)$, then prediction will be

$$\mathbf{f}(\mathbf{x}(t), \mathbf{g}(\mathbf{x}(t), \mathbf{g}(\mathbf{x}(t-1), \mathbf{g}(\mathbf{x}(t-2), \mathbf{c}(t-2) | \mathbf{W}_g) | \mathbf{W}_g) | \mathbf{W}_g) | \mathbf{W}_f)$$

- it should be clear that the gradients of this with respect to the weights can be found with the chain rule
- The back-propagated error will involve applying $\mathbf{f}$ multiple times
- Each time the error will get multiplied by some factor a
- If $\mathbf{y}(t)$ depends on the input $\mathbf{x}(t-\tau)$ then the back-propagated signal will be proportional to $a^{\tau-1}$
- This either vanishes or explodes when τ becomes large

Vanishing and Exploding Gradients

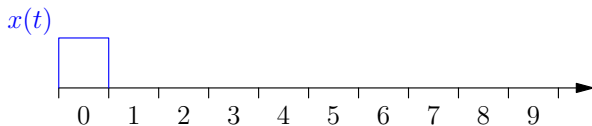
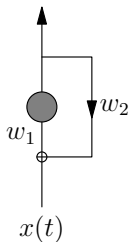
$$y(t) = w_1 (x(t) + w_2 y(t-1))$$



Vanishing and Exploding Gradients

$$y(t) = w_1 (x(t) + w_2 y(t-1))$$

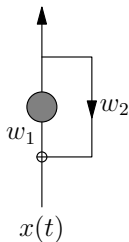
$$w_1 = w_2 = 0.9$$



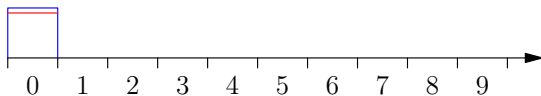
Vanishing and Exploding Gradients

$$y(t) = w_1 (x(t) + w_2 y(t-1))$$

$$w_1 = w_2 = 0.9$$



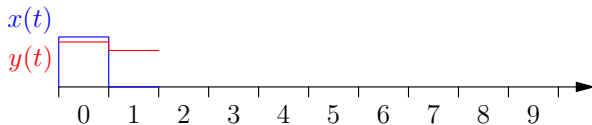
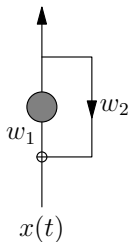
$x(t)$
 $y(t)$



Vanishing and Exploding Gradients

$$y(t) = w_1 (x(t) + w_2 y(t-1))$$

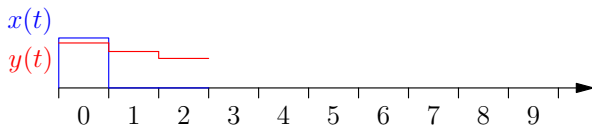
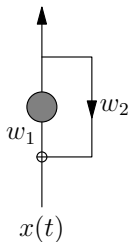
$$w_1 = w_2 = 0.9$$



Vanishing and Exploding Gradients

$$y(t) = w_1 (x(t) + w_2 y(t-1))$$

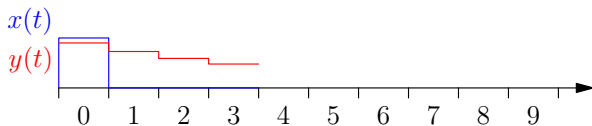
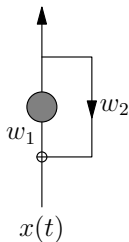
$$w_1 = w_2 = 0.9$$



Vanishing and Exploding Gradients

$$y(t) = w_1 (x(t) + w_2 y(t-1))$$

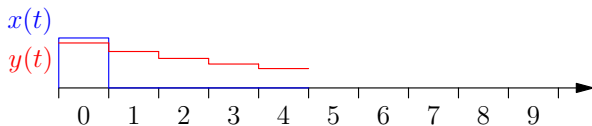
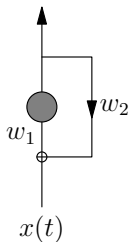
$$w_1 = w_2 = 0.9$$



Vanishing and Exploding Gradients

$$y(t) = w_1 (x(t) + w_2 y(t-1))$$

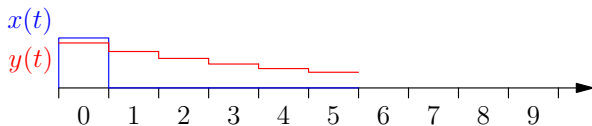
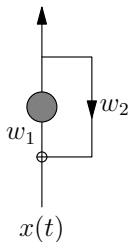
$$w_1 = w_2 = 0.9$$



Vanishing and Exploding Gradients

$$y(t) = w_1 (x(t) + w_2 y(t-1))$$

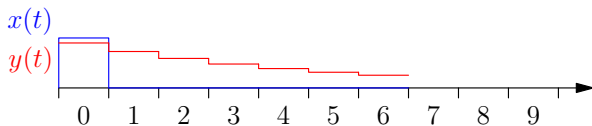
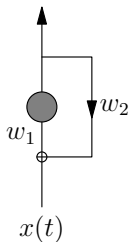
$$w_1 = w_2 = 0.9$$



Vanishing and Exploding Gradients

$$y(t) = w_1 (x(t) + w_2 y(t-1))$$

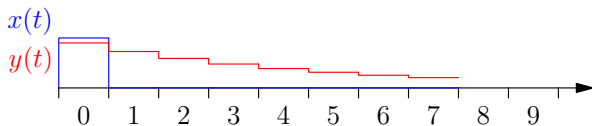
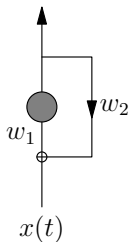
$$w_1 = w_2 = 0.9$$



Vanishing and Exploding Gradients

$$y(t) = w_1 (x(t) + w_2 y(t-1))$$

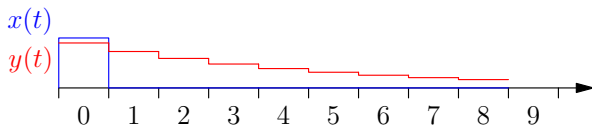
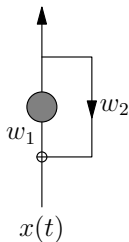
$$w_1 = w_2 = 0.9$$



Vanishing and Exploding Gradients

$$y(t) = w_1 (x(t) + w_2 y(t-1))$$

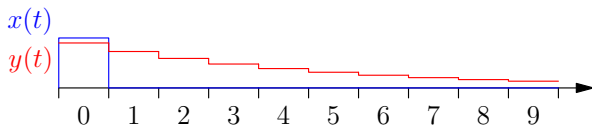
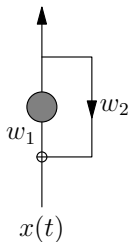
$$w_1 = w_2 = 0.9$$



Vanishing and Exploding Gradients

$$y(t) = w_1 (x(t) + w_2 y(t-1))$$

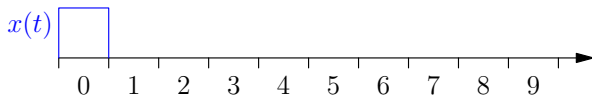
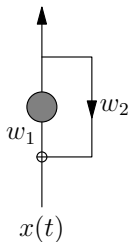
$$w_1 = w_2 = 0.9$$



Vanishing and Exploding Gradients

$$y(t) = w_1 (x(t) + w_2 y(t-1))$$

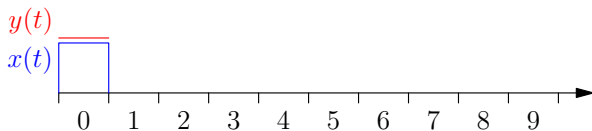
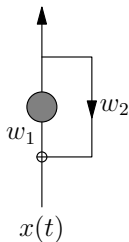
$$w_1 = w_2 = 1.1$$



Vanishing and Exploding Gradients

$$y(t) = w_1 (x(t) + w_2 y(t-1))$$

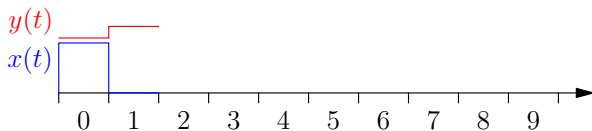
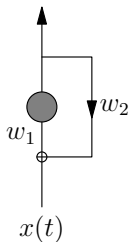
$$w_1 = w_2 = 1.1$$



Vanishing and Exploding Gradients

$$y(t) = w_1 (x(t) + w_2 y(t-1))$$

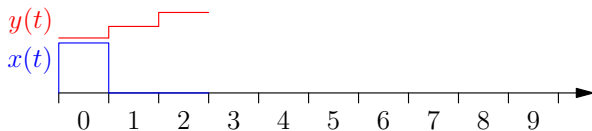
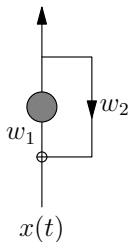
$$w_1 = w_2 = 1.1$$



Vanishing and Exploding Gradients

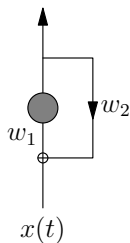
$$y(t) = w_1 (x(t) + w_2 y(t-1))$$

$$w_1 = w_2 = 1.1$$

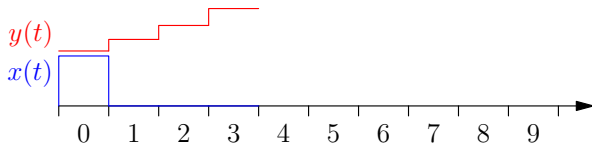


Vanishing and Exploding Gradients

$$y(t) = w_1 (x(t) + w_2 y(t-1))$$

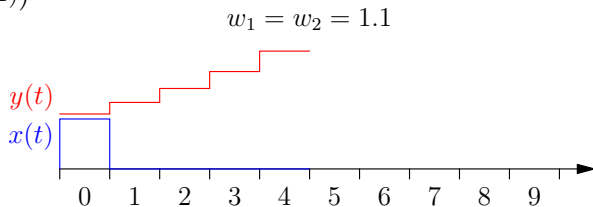
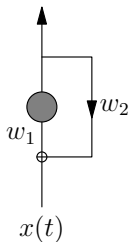


$$w_1 = w_2 = 1.1$$



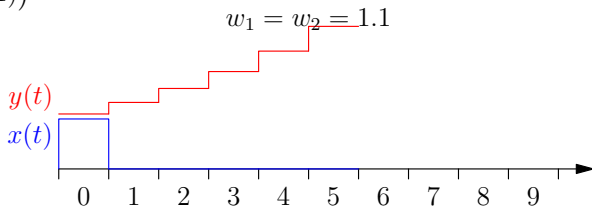
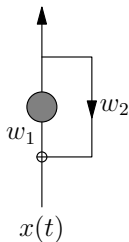
Vanishing and Exploding Gradients

$$y(t) = w_1 (x(t) + w_2 y(t-1))$$



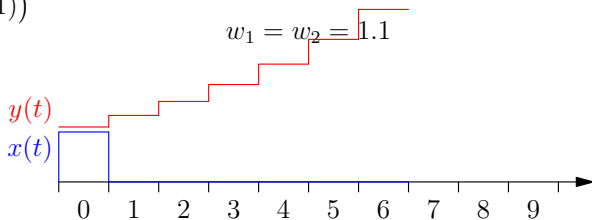
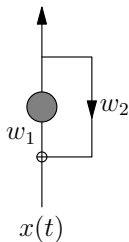
Vanishing and Exploding Gradients

$$y(t) = w_1 (x(t) + w_2 y(t-1))$$



Vanishing and Exploding Gradients

$$y(t) = w_1 (x(t) + w_2 y(t-1))$$



- The LSTM (long-short term memory) was designed to solve this problem

- The LSTM (long-short term memory) was designed to solve this problem
- Key ideas: to retain a 'long-term memory' requires

$$\mathbf{c}(t) = \mathbf{c}(t - 1)$$

- The LSTM (long-short term memory) was designed to solve this problem
- Key ideas: to retain a 'long-term memory' requires

$$\mathbf{c}(t) = \mathbf{c}(t - 1)$$

- Sometimes we have to forget and sometimes we have to change a memory

- The LSTM (long-short term memory) was designed to solve this problem
- Key ideas: to retain a 'long-term memory' requires

$$\mathbf{c}(t) = \mathbf{c}(t - 1)$$

- Sometimes we have to forget and sometimes we have to change a memory
- To do this we should use 'gates' that saturate at 0 and 1

- The LSTM (long-short term memory) was designed to solve this problem
- Key ideas: to retain a 'long-term memory' requires

$$\mathbf{c}(t) = \mathbf{c}(t - 1)$$

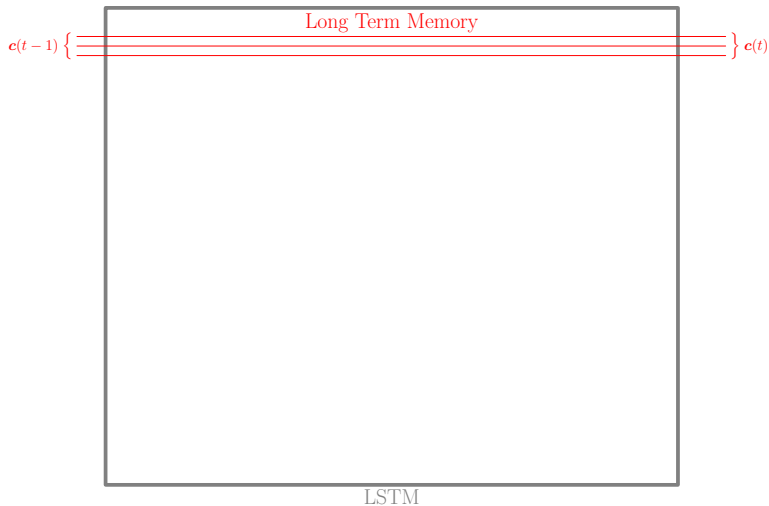
- Sometimes we have to forget and sometimes we have to change a memory
- To do this we should use 'gates' that saturate at 0 and 1
- Sigmoid functions naturally saturate at 0 and 1

LSTM Architecture

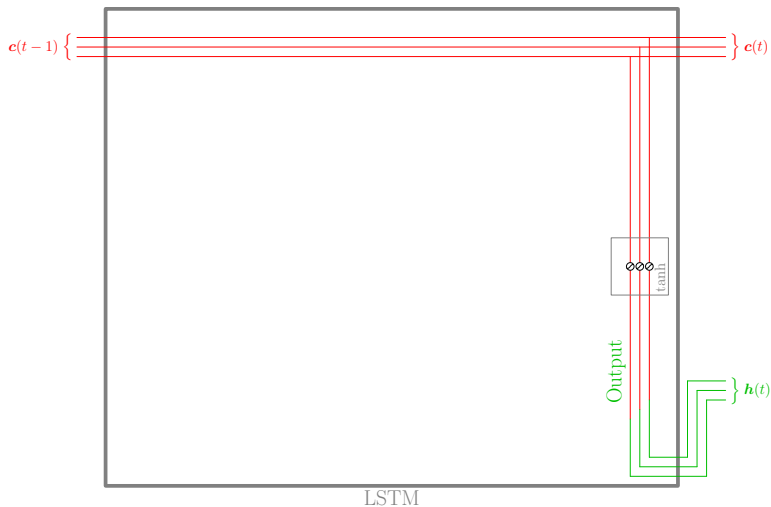


LSTM

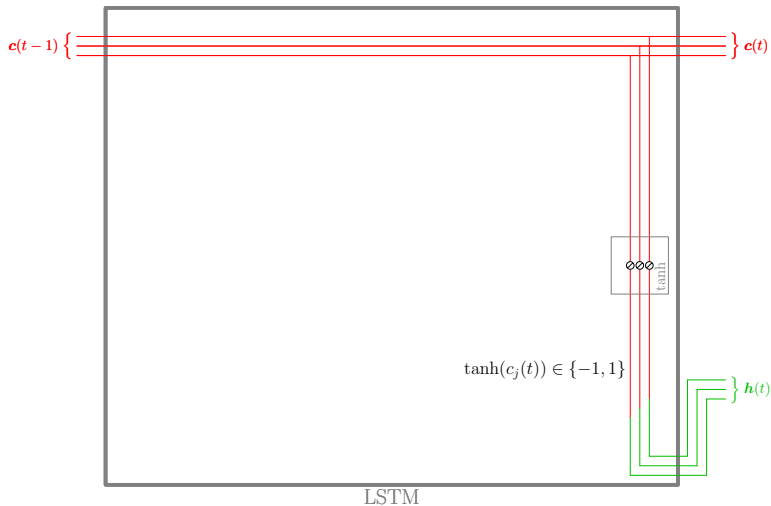
LSTM Architecture



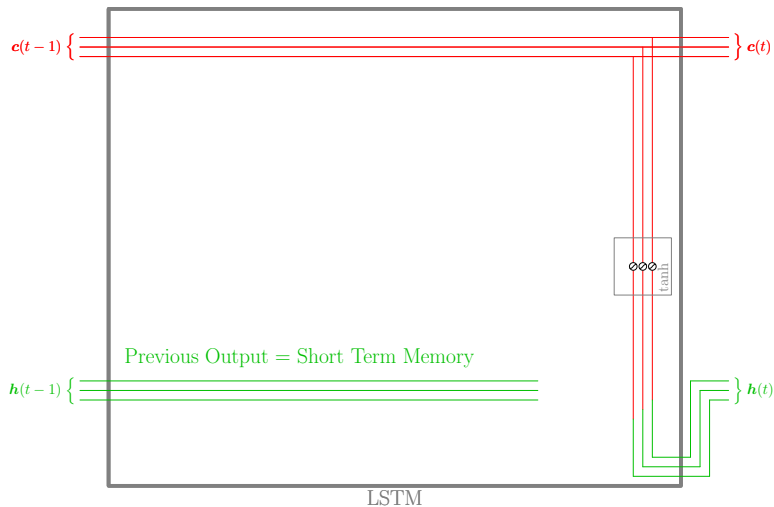
LSTM Architecture



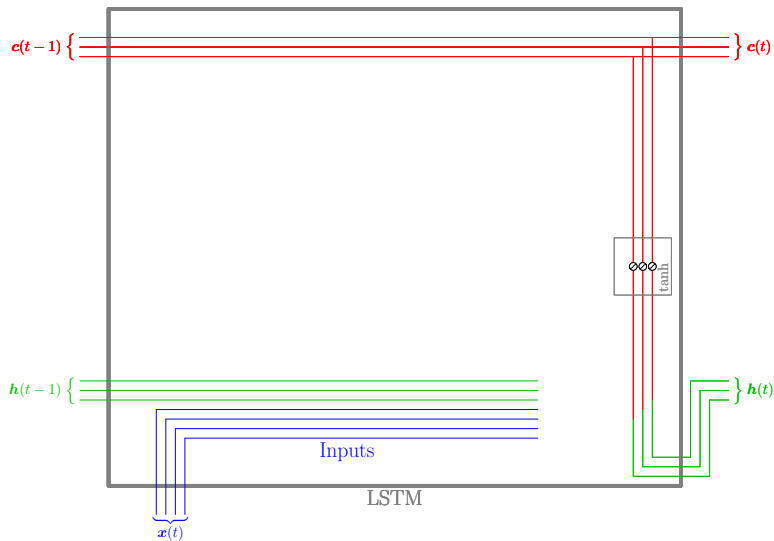
LSTM Architecture



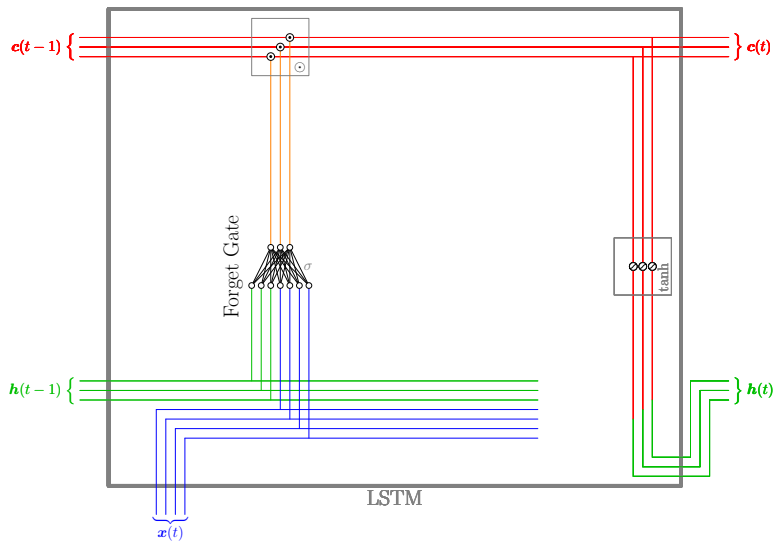
LSTM Architecture



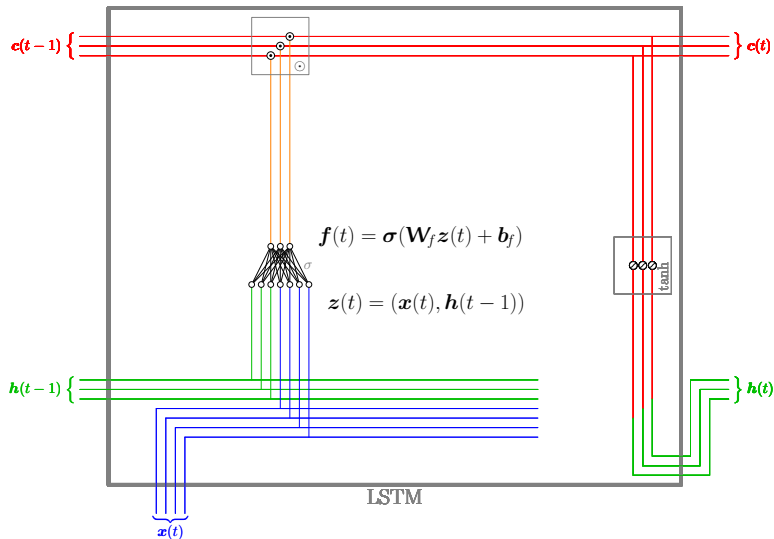
LSTM Architecture



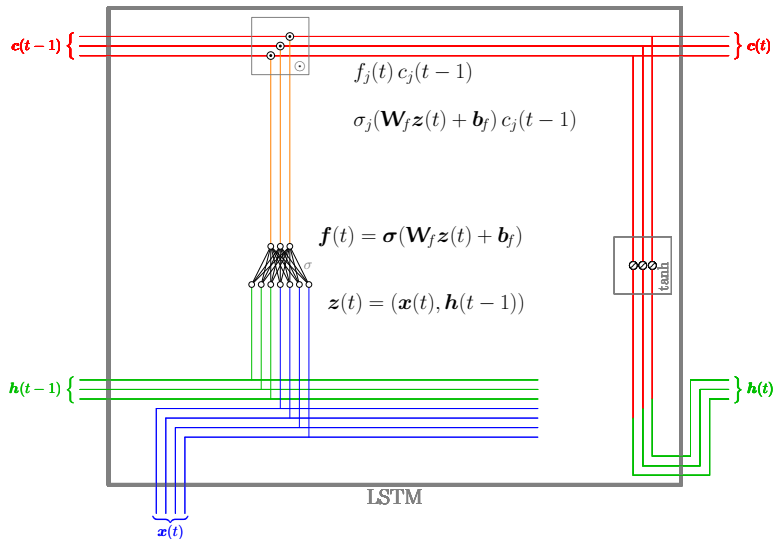
LSTM Architecture



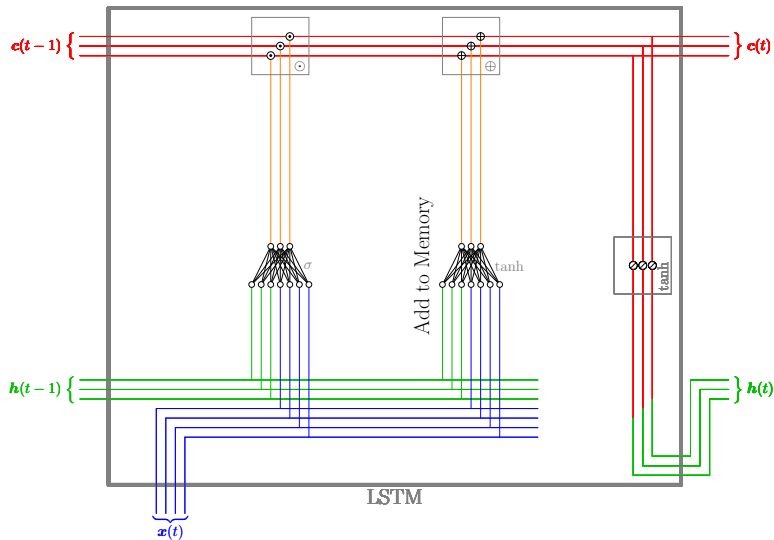
LSTM Architecture



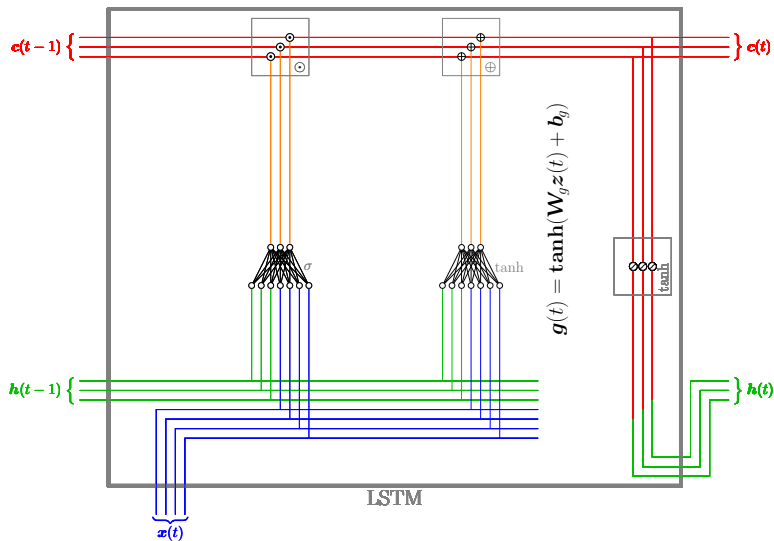
LSTM Architecture



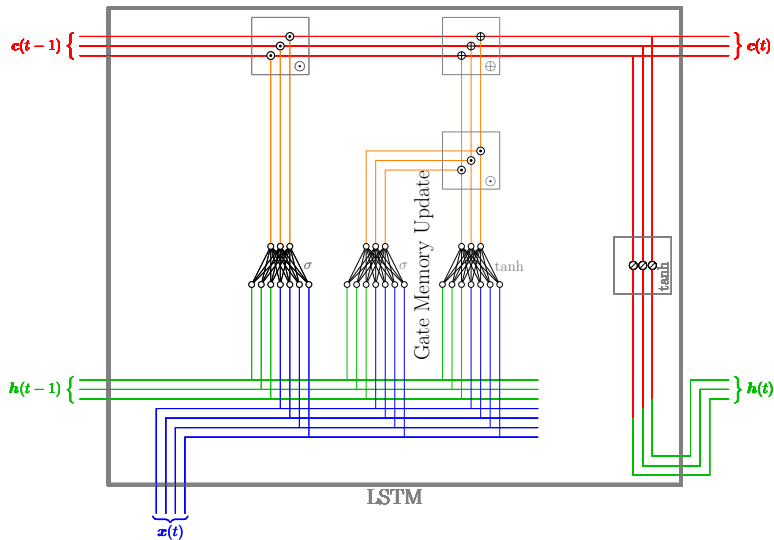
LSTM Architecture



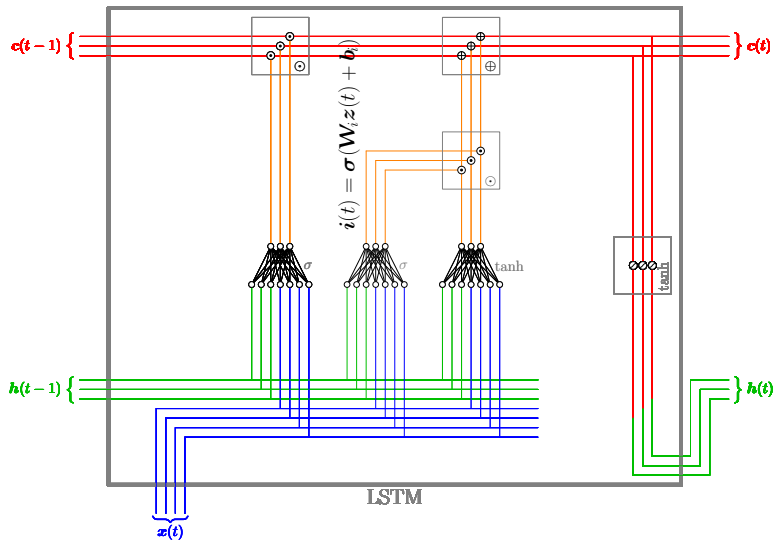
LSTM Architecture



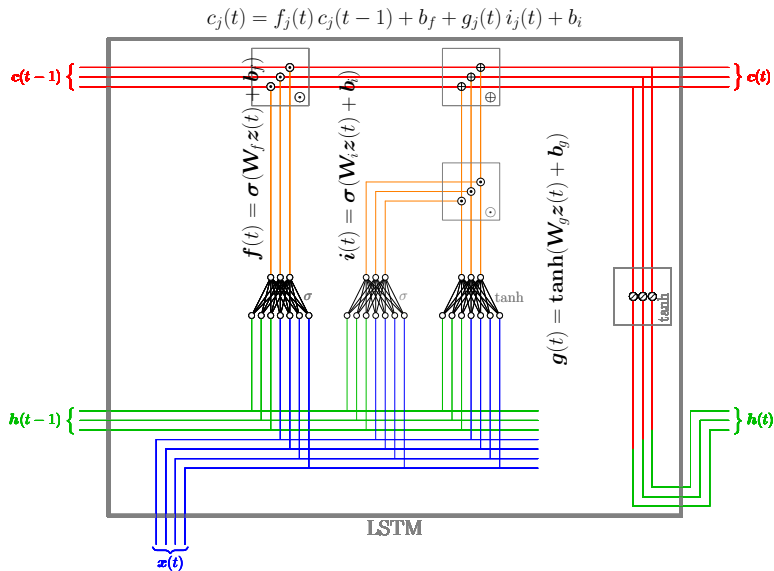
LSTM Architecture



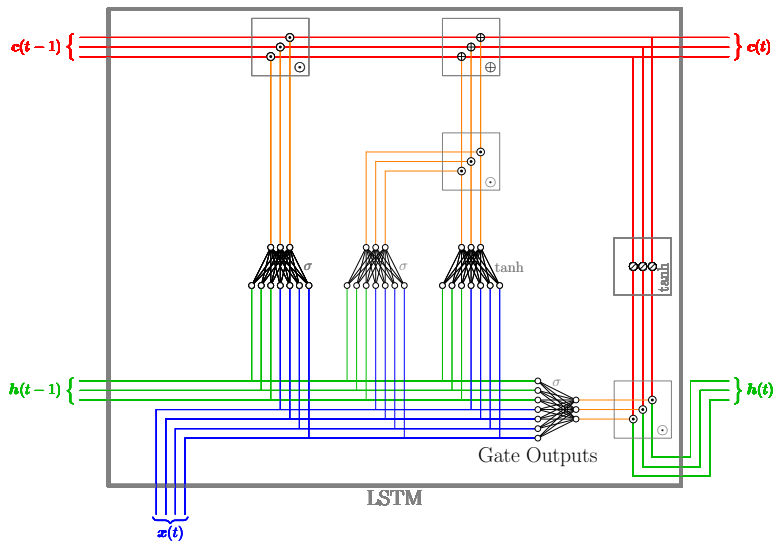
LSTM Architecture



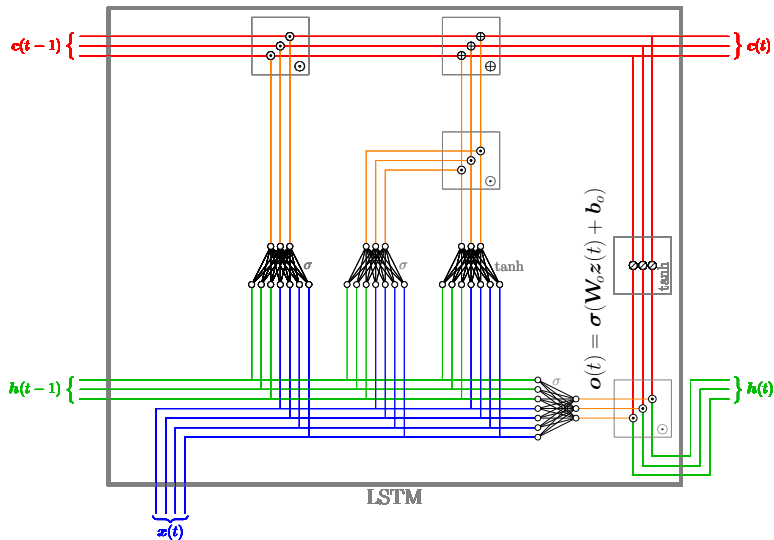
LSTM Architecture



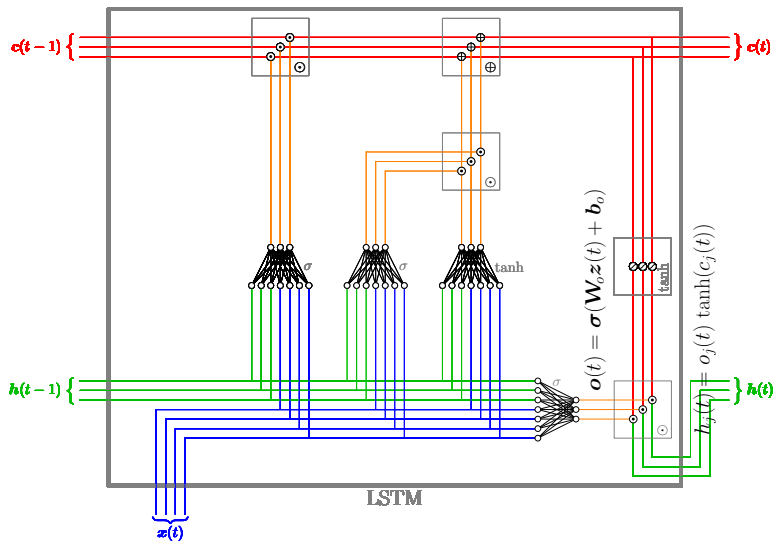
LSTM Architecture



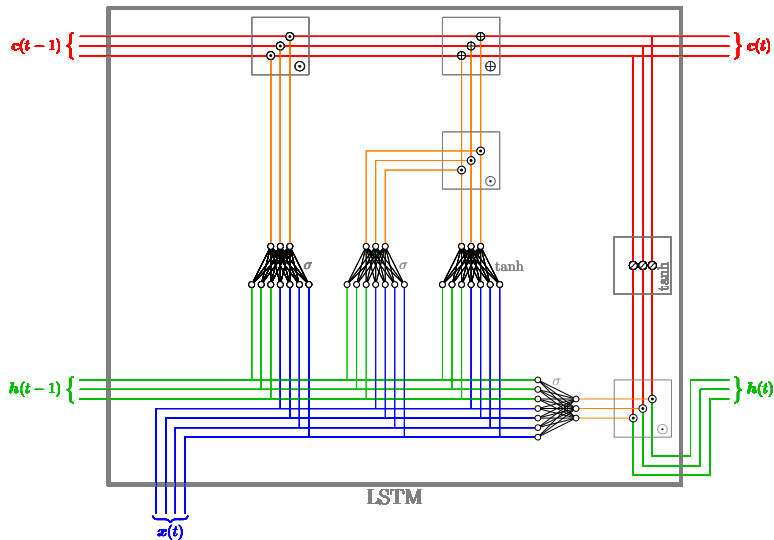
LSTM Architecture



LSTM Architecture



LSTM Architecture



Update Equations

Initially, for $t = 1$, $\mathbf{h}(0) = \mathbf{0}$

- Inputs $\mathbf{z}(t) = (\mathbf{x}(t), \mathbf{h}(t - 1))$

Update Equations

Initially, for $t = 1$, $\mathbf{h}(0) = \mathbf{0}$

- Inputs $\mathbf{z}(t) = (\mathbf{x}(t), \mathbf{h}(t-1))$
- Network updates ($\mathbf{W}_*$ and $\mathbf{b}_*$ are the learnable parameters)

$$\begin{aligned} \mathbf{f}(t) &= \sigma(\mathbf{W}_f \mathbf{z}(t) + \mathbf{b}_f) & \mathbf{i}(t) &= \sigma(\mathbf{W}_i \mathbf{z}(t) + \mathbf{b}_i) \\ \mathbf{g}(t) &= \tanh(\mathbf{W}_g \mathbf{z}(t) + \mathbf{b}_g) & \mathbf{o}(t) &= \sigma(\mathbf{W}_o \mathbf{z}(t) + \mathbf{b}_o) \end{aligned}$$

Update Equations

Initially, for $t = 1$, $\mathbf{h}(0) = \mathbf{0}$

- Inputs $\mathbf{z}(t) = (\mathbf{x}(t), \mathbf{h}(t-1))$
- Network updates ($\mathbf{W}_*$ and $\mathbf{b}_*$ are the learnable parameters)

$$\begin{aligned}\mathbf{f}(t) &= \sigma(\mathbf{W}_f \mathbf{z}(t) + \mathbf{b}_f) & \mathbf{i}(t) &= \sigma(\mathbf{W}_i \mathbf{z}(t) + \mathbf{b}_i) \\ \mathbf{g}(t) &= \tanh(\mathbf{W}_g \mathbf{z}(t) + \mathbf{b}_g) & \mathbf{o}(t) &= \sigma(\mathbf{W}_o \mathbf{z}(t) + \mathbf{b}_o)\end{aligned}$$

- Long-term memory update

$$\mathbf{c}(t) = \mathbf{f}(t) \odot \mathbf{c}(t-1) + \mathbf{g}(t) \odot \mathbf{i}(t)$$

Update Equations

Initially, for $t = 1$, $\mathbf{h}(0) = \mathbf{0}$

- Inputs $\mathbf{z}(t) = (\mathbf{x}(t), \mathbf{h}(t-1))$
- Network updates ($\mathbf{W}_*$ and $\mathbf{b}_*$ are the learnable parameters)

$$\begin{aligned}\mathbf{f}(t) &= \sigma(\mathbf{W}_f \mathbf{z}(t) + \mathbf{b}_f) & \mathbf{i}(t) &= \sigma(\mathbf{W}_i \mathbf{z}(t) + \mathbf{b}_i) \\ \mathbf{g}(t) &= \tanh(\mathbf{W}_g \mathbf{z}(t) + \mathbf{b}_g) & \mathbf{o}(t) &= \sigma(\mathbf{W}_o \mathbf{z}(t) + \mathbf{b}_o)\end{aligned}$$

- Long-term memory update

$$\mathbf{c}(t) = \mathbf{f}(t) \odot \mathbf{c}(t-1) + \mathbf{g}(t) \odot \mathbf{i}(t)$$

- Output $\mathbf{h}(t) = \mathbf{o}(t) \odot \tanh(\mathbf{c}(t))$

- We can train an LSTM by unwrapping it in time.

- We can train an LSTM by unwrapping it in time.
- Note that it involves four dense layers with sigmoidal (or tanh) outputs.

- We can train an LSTM by unwrapping it in time.
- Note that it involves four dense layers with sigmoidal (or tanh) outputs.
- This means that typically it is very slow to train.

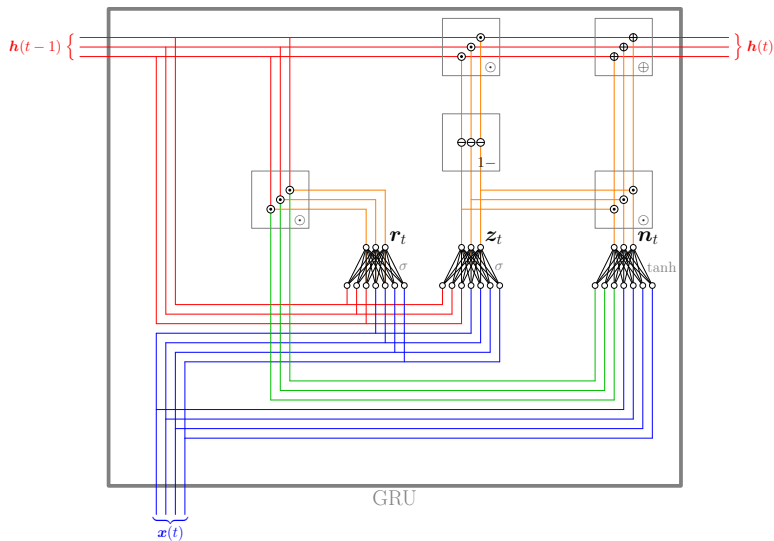
- We can train an LSTM by unwrapping it in time.
- Note that it involves four dense layers with sigmoidal (or tanh) outputs.
- This means that typically it is very slow to train.
- There are a few variants of LSTMs, but all are very similar. The most popular is probably the Gated Recurrent Unit (GRU).

LSTM Success Stories

- LSTMs have been used to win many competitions in speech and handwriting recognition.
- Major technology companies including Google, Apple, and Microsoft are using LSTMs as fundamental components in products.
- Google used LSTM for speech recognition on the smartphone, for Google Translate.
- Apple uses LSTM for the "Quicktype" function on the iPhone and for Siri.
- Amazon uses LSTM for Amazon Alexa.
- In 2017, Facebook performed some 4.5 billion automatic translations every day using long short-term memory networks¹.

¹https://en.wikipedia.org/wiki/Long_short-term_memory

Gated Recurrent Unit (GRU)



Gated Recurrent Unit (GRU)

- $\mathbf{x}(t)$: input vector
- $\mathbf{h}(t)$: output vector (and 'hidden state')
- $\mathbf{r}(t)$: reset gate vector
- $\mathbf{z}(t)$: update gate vector
- $\mathbf{n}(t)$: new state vector (before update is applied)
- $\mathbf{W}$ and $\mathbf{b}$: parameter matrices and biases

Gated Recurrent Unit (GRU)

Initially, for $t = 1$, $\mathbf{h}(0) = \mathbf{0}$

$$\mathbf{z}(t) = \sigma(\mathbf{W}_z(\mathbf{x}(t), \mathbf{h}(t-1)) + \mathbf{b}_z)$$

$$\mathbf{r}(t) = \sigma(\mathbf{W}_r(\mathbf{x}(t), \mathbf{h}(t-1)) + \mathbf{b}_r)$$

$$\mathbf{n}(t) = \tanh(\mathbf{W}_n(\mathbf{x}(t), \mathbf{r}(t) \odot \mathbf{h}(t-1)) + \mathbf{b}_h)$$

$$\mathbf{h}(t) = (1 - \mathbf{z}(t)) \odot \mathbf{h}(t-1) + \mathbf{z}(t) \odot \mathbf{n}(t)$$

Most implementations follow the original paper and swap $(1 - \mathbf{z}(t))$ and $(\mathbf{z}(t))$ in the $\mathbf{h}(t)$ update; this doesn't change the operation of the network, but does change the interpretation of the update gate, as the gate would have to produce a 0 when an update was to occur, and a 1 when no update is to happen (which is somewhat counter-intuitive)!

GRU or LSTM?

- GRUs have two gates (reset and update) whereas LSTM has three gates (input/output/forget)
- GRU performance on par with LSTM but computationally more efficient (less operations & weights).
- In general, if you have a very large dataset then LSTMs will likely perform slightly better.
- GRUs are a good choice for smaller datasets.